

# Lecture 01: Introduction to OOP

## Kotlin Object-Oriented Programming

# Agenda

1. Why object-oriented programming?
2. Core ideas: object, class, state, behaviour
3. Creating a class and an object in Kotlin
4. Properties, methods, and a first design sketch

# From scripts to structure

Intro Kotlin covered expressions, control flow, and collections. Larger programs need **structure**:

- Group related data with the operations that use it
- Hide internal details; expose a clear interface
- Reuse and extend behaviour without copy-paste

**Object-oriented programming (OOP)** organises software around *objects* — bundles of state + behaviour — described by *classes*.

# Four classic pillars (preview)

- **Encapsulation** — bundle + control access
- **Abstraction** — show what, hide how
- **Inheritance** — specialise a type
- **Polymorphism** — one interface, many forms

We introduce the vocabulary now; later lectures deepen each pillar with Kotlin's rules (open, override, visibility, ...).

# Class vs object

## Class

A blueprint: name, properties (data), and member functions (behaviour).

## Object (instance)

A concrete value created from that blueprint, living in memory, with its own property values.

Analogy: `Student` is the class; `Student("Ada", 2024)` is an object.

# State and behaviour

- **State** — properties: `name`, `balance`, `items`
- **Behaviour** — functions that read/update state or compute results
- Prefer modelling *domain nouns* as classes (`Account`, `Order`, `Sensor`)

## Common mistake

Putting all logic in `main` with loose variables. If several values always travel together, they probably belong in a class.

## Quiz 1 — Class vs object

### Quiz

Which statement is accurate?

1. A class is a runtime value; an object is only a compile-time name
2. A class describes a type; each object is an instance of that type
3. In Kotlin you cannot create more than one object per class
4. Objects exist only inside `enum` class

*Pause — answer (clicker / raise hand / neighbour) before the next slide.*

### Answer

2. A class describes a type; each object is an instance of that type.

You may create many instances. Classes are blueprints; objects are the runtime values.

# Minimal class

```
1 class Person {  
2     var name: String = "Unknown"  
3     var age: Int = 0  
4  
5     fun introduce() {  
6         println("I am $name, age $age")  
7     }  
8 }  
9  
10 fun main() {  
11     val p = Person()    // create an object  
12     p.name = "Ada"  
13     p.age = 36  
14     p.introduce()  
15 }
```

# Creating an instance

- Syntax: `ClassName(...)` — looks like a function call
- Default (no primary constructor args yet): `Person()`
- Reference type: `val p = Person()` — `p` refers to the object
- `val` prevents rebinding `p`; the object's `var` properties may still change

## Try this

Create two `Person` instances with different names; call `introduce()` on both.

## Multiple instances, separate state

```
1 fun main() {  
2     val a = Person()  
3     val b = Person()  
4     a.name = "Ada"  
5     b.name = "Grace"  
6     a.introduce()    // Ada  
7     b.introduce()    // Grace  
8 }
```

Changing a does not change b — each object has its own property storage.

## Member functions use the receiver

```
1 class Counter {  
2     var value: Int = 0  
3     fun inc() { value += 1 }  
4     fun reset() { value = 0 }  
5 }  
6  
7 fun main() {  
8     val c = Counter()  
9     c.inc(); c.inc()  
10    println(c.value)    // 2  
11 }
```

Inside `inc`, `value` refers to *this* object's field (explicit `this` comes in Lecture 02).

# Design sketch: nouns and verbs

Before coding a module, list:

1. Domain nouns → candidate classes
2. Verbs on those nouns → methods
3. Data each noun must remember → properties

Example: banking — Account with `balance`; methods `deposit`, `withdraw`. We return to this in later lectures (inheritance practice: accountant).

## Quiz 2 — What prints?

### Quiz

What is printed?

```
1 class Box { var n = 1 }  
2 fun main() {  
3     val x = Box()  
4     val y = x  
5     y.n = 5  
6     println(x.n)  
7 }
```

- 1.
- 5
- Compile error
- null

*Pause — answer (clicker / raise hand / neighbour) before the next slide.*

Answer

2. 5

`x` and `y` refer to the *same* object. Updating via `y` is visible through `x`. (Rebinding is different from mutating fields.)

# Summary

- OOP structures programs as interacting objects
- A **class** is a blueprint; an **object** is an instance
- Properties hold state; member functions provide behaviour
- `ClassName()` creates an instance; references can alias the same object

# Looking ahead

Next: functions as members and top-level — parameters, `this`, named arguments, and return types.